

Chit 11

Database Trigger (Row level, After Trigger). Write a database trigger on Library table. The System should keep track of the records that are being updated or deleted. The old value of updated or deleted records should be added in Library_Audit table.

```
CREATE TABLE Library1 (  
    Book_ID NUMBER,  
    Book_Name VARCHAR2(50),  
    Author VARCHAR2(50),  
    Price NUMBER  
);
```

```
CREATE TABLE Library_Audit1 (  
    Book_ID NUMBER,  
    Book_Name VARCHAR2(50),  
    Author VARCHAR2(50),  
    Price NUMBER,  
    Action VARCHAR2(10),  
    Action_Date DATE  
);
```

```
INSERT INTO Library1 VALUES (1, 'DBMS', 'Korth', 500);  
INSERT INTO Library1 VALUES (2, 'OS', 'Galvin', 600);  
INSERT INTO Library1 VALUES (3, 'CN', 'Tanenbaum', 550);
```

```
CREATE OR REPLACE TRIGGER trg_library_audit_after  
AFTER UPDATE OR DELETE ON Library  
FOR EACH ROW  
BEGIN  
    -- For UPDATE: store OLD values  
    IF UPDATING THEN  
        INSERT INTO Library_Audit  
        (Book_ID, Book_Name, Author, Price, Action, Action_Date)  
        VALUES  
        (:OLD.Book_ID, :OLD.Book_Name, :OLD.Author, :OLD.Price, 'UPDATE', SYSDATE);  
  
    -- For DELETE: store OLD values  
    ELSIF DELETING THEN  
        INSERT INTO Library_Audit  
        (Book_ID, Book_Name, Author, Price, Action, Action_Date)  
        VALUES  
        (:OLD.Book_ID, :OLD.Book_Name, :OLD.Author, :OLD.Price, 'DELETE', SYSDATE);  
    END IF;  
END;
```

```

/
select * from library1;

select * from library_audit1;

delete from library1 where book_name='DBMS';

update library1 set book_name='Marathi' where book_id=2;

select * from library1;

select * from library_audit1;

```

Chit 10

Database Trigger (Row level, Before Triggers). Write a database trigger on Library table. The System should keep track of the records that are being updated or deleted. The old value of updated or deleted records should be added in Library_Audit table.

```

CREATE TABLE Library2 (
    Book_ID NUMBER,
    Book_Name VARCHAR2(50),
    Author VARCHAR2(50),
    Price NUMBER
);

CREATE TABLE Library_Audit2 (
    Book_ID NUMBER,
    Book_Name VARCHAR2(50),
    Author VARCHAR2(50),
    Price NUMBER,
    Action VARCHAR2(10),
    Action_Date DATE
);

INSERT INTO Library2 VALUES (1, 'DBMS', 'Korth', 500);
INSERT INTO Library2 VALUES (2, 'OS', 'Galvin', 600);
INSERT INTO Library2 VALUES (3, 'CN', 'Tanenbaum', 550);

CREATE OR REPLACE TRIGGER trg_library_audit_before2
BEFORE UPDATE OR DELETE ON Library2
FOR EACH ROW
BEGIN
    -- For UPDATE operation

```

```

IF UPDATING THEN
    INSERT INTO Library_Audit2
    (Book_ID, Book_Name, Author, Price, Action, Action_Date)
    VALUES
    (:OLD.Book_ID, :OLD.Book_Name, :OLD.Author, :OLD.Price, 'UPDATE', SYSDATE);

    -- For DELETE operation
ELSIF DELETING THEN
    INSERT INTO Library_Audit2
    (Book_ID, Book_Name, Author, Price, Action, Action_Date)
    VALUES
    (:OLD.Book_ID, :OLD.Book_Name, :OLD.Author, :OLD.Price, 'DELETE', SYSDATE);
END IF;
END;
/

```

```
select * from library2;
```

```
delete from library2 where book_name='DBMS';
```

```
delete from library2 where book_name='DBMS';
```

Chit 2

Unnamed PL/SQL code block: Use of Control structure and Exception handling is mandatory. Write a PL/SQL block of code for the following requirements:- Schema: 1. Borrower(Rollin, Name, DateofIssue, NameofBook, Status) 2. Fine(Roll_no,Date,Amt) Accept roll_no & name of book from user. Check the number of days (from date of issue), if days are between 15 to 30 then fine amount will be Rs 5per day. If no. of days>30, per day fine will be Rs 50 per day & for days less than 30, Rs. 5 per day. After submitting the book, status will change from I to R. If condition of fine is true, then details will be stored into fine table.

```

CREATE TABLE Borrower (
    Rollin    NUMBER,
    Names     VARCHAR2(50),
    DateofIssue DATE,
    NameofBook VARCHAR2(50),
    Status    CHAR(1)    -- 'I' = Issued, 'R' = Returned
);

```

```

CREATE TABLE Fine (
    Roll_no NUMBER,
    Date1   DATE,
    Amt     NUMBER
);

```

```
INSERT INTO Borrower VALUES (1, 'Amit', DATE '2024-03-01', 'DBMS', 'I');
INSERT INTO Borrower VALUES (2, 'Neha', DATE '2024-03-10', 'OS', 'I');
INSERT INTO Borrower VALUES (3, 'Sneha', DATE '2024-03-10', 'DS', 'I');
```

```
INSERT INTO Fine (Roll_no, Date1, Amt)
VALUES (1, SYSDATE, 100);
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  v_roll  Borrower.Rollin%TYPE;
  v_book  Borrower.NameofBook%TYPE;
  v_date  Borrower.DateofIssue%TYPE;
  v_days  NUMBER;
  v_fine  NUMBER := 0;
```

```
BEGIN
```

```
  -- Accept input
```

```
  v_roll := &roll_no;
```

```
  v_book := '&book_name';
```

```
  -- Fetch issue date
```

```
  SELECT DateofIssue
```

```
  INTO v_date
```

```
  FROM Borrower
```

```
  WHERE Rollin = v_roll
```

```
    AND NameofBook = v_book
```

```
    AND Status = 'I';
```

```
  -- Calculate number of days
```

```
  v_days := TRUNC(SYSDATE - v_date);
```

```
  -- Fine calculation (control structure)
```

```
  IF v_days BETWEEN 15 AND 30 THEN
```

```
    v_fine := (v_days - 15) * 5;
```

```
  ELSIF v_days > 30 THEN
```

```
    v_fine := (15 * 5) + (v_days - 30) * 50;
```

```
  ELSE
```

```
    v_fine := 0;
```

```
  END IF;
```

```
  -- Update status to Returned
```

```
  UPDATE Borrower
```

```
  SET Status = 'R'
```

```

WHERE Rollin = v_roll
AND NameofBook = v_book;

-- Insert fine if applicable
IF v_fine > 0 THEN
    INSERT INTO Fine VALUES (v_roll, SYSDATE, v_fine);
END IF;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Days: ' || v_days);
DBMS_OUTPUT.PUT_LINE('Fine Amount: ' || v_fine);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No matching issued record found');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

select * from borrower;

```

Chit 3

Unnamed PL/SQL code block: Use of Control structure and Exception handling is mandatory. Write a PL/SQL block of code for the following requirements: -Consider table Stud(Roll, Att,Status) Write a PL/SQL block for following requirement and handle the exceptions. Roll no. of student will be entered by user. Attendance of roll no. entered by user will be checked in Stud table. If attendance is less than 75% then display the message “Term not granted” and set the status in stud table as “D”. Otherwise display message “Term granted” and set the status in stud table as “ND” .

```

CREATE TABLE Stud (
    Roll NUMBER,
    Att NUMBER,
    Status VARCHAR2(5)
);

INSERT INTO Stud VALUES (1, 80, NULL);
INSERT INTO Stud VALUES (2, 60, NULL);
INSERT INTO Stud VALUES (3, 75, NULL);

SET SERVEROUTPUT ON;

```

```

DECLARE
    v_roll Stud.Roll%TYPE;
    v_att Stud.Att%TYPE;

BEGIN

    v_roll := &roll_no;

    SELECT Att INTO v_att
    FROM Stud
    WHERE Roll = v_roll;

    IF v_att < 75 THEN
        DBMS_OUTPUT.PUT_LINE('Term not granted');

        UPDATE Stud
        SET Status = 'D'
        WHERE Roll = v_roll;

    ELSE
        DBMS_OUTPUT.PUT_LINE('Term granted');

        UPDATE Stud
        SET Status = 'ND'
        WHERE Roll = v_roll;
    END IF;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);
END;
/

```